Hapimatic on Windows with WSL2

Beta guide for ESE5180 teams, from Matt Starfield, the Senior Electrical Engineer at Solid Product Design · tested end to end on a Windows 11 laptop on Oct 25–26, 2026

What this setup ran, start to finish: Hapimatic Core and Claude Code inside Ubuntu on WSL2; Saleae Logic 2's MCP server on Windows, reached from WSL; a PPK2 powering and measuring a test board; firmware built and flashed from WSL; and Claude Code running the whole loop on its own from one requirement. Windows keeps every USB device. WSL calls the Windows tools.

1 Ubuntu on WSL2, with systemd

```
wsl --install -d Ubuntu-24.04        (PowerShell, then reboot)
```

In Ubuntu, make sure `/etc/wsl.conf` has:

```
[boot]
systemd=true
```

```
sudo apt-get install -y python3 jq tmux git curl
```

2 Keep Ubuntu awake and on localhost

Create `%USERPROFILE%\.wslconfig` on Windows, then run `wsl --shutdown`:

```
[wsl2]
networkingMode=mirrored
vmIdleTimeout=-1

[general]
instanceIdleTimeout=-1
```

WHY Without it, WSL stopped Ubuntu (and Hapimatic) 90 s after the last terminal closed. Mirrored mode lets WSL reach Logic 2 on `127.0.0.1`

3 An agent CLI, then Hapimatic Core

The installer's check needs Claude Code installed, even if you use Codex:

```
curl -fsSL https://claude.ai/install.sh | bash
claude                       # sign in, if you have a plan
curl -fsSL https://github.com/SolidPD/hapimatic-
core/releases/latest/download/install-hapimatic-v1.sh | bash
```

Then set up the two systemd user services exactly as in the install guide (*Keep the server running*), and:

```
sudo loginctl enable-linger "$USER"
```

4 Sign in from the laptop's browser

Open `http://localhost:3006` in Edge or Chrome on Windows. Your access token is in Ubuntu:

```
jq -r .cliApiToken ~/.hapi/settings.json
```

TIP Sign in on the laptop, not your phone: on iPhone Chrome the token field can hide behind the keyboard (we are fixing it)
From your phone later: add Tailscale on both devices, then open `http://<laptop-tailscale-ip>:3006`

5 The Saleae, through Logic 2's MCP server

Logic 2 (2.4.48 or newer) on **Windows** › Settings › Automation › turn on **MCP Server**. In Ubuntu:

```
claude mcp add --transport http -s user logic2
http://127.0.0.1:10530
claude mcp list              # logic2: Connected
```

User scope reaches every Hapimatic session too. Logic 2 saves exports to Windows paths: `C:\tmp\cap` is `/mnt/c/tmp/cap` in WSL
ORIGINAL LOGIC 4 MS/s worked; 12 MS/s timed out while the PPK2 streamed, and its threshold is fixed

6 The PPK2, through Windows Python

Install Python 3 on Windows, then in PowerShell:

```
py -m venv $HOME\ppk2-venv
$HOME\ppk2-venv\Scripts\pip install ppk2-api==0.9.2
```

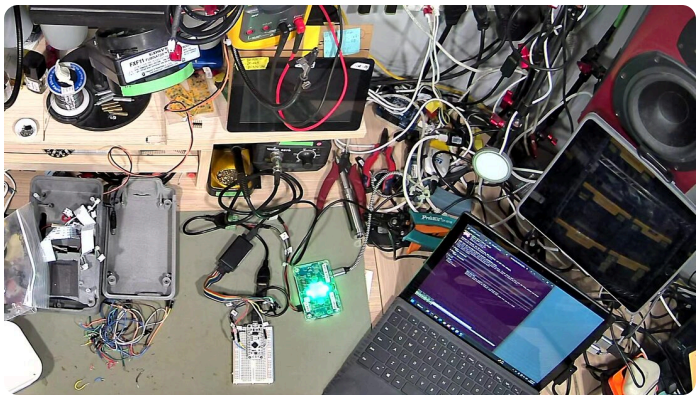
From WSL, run that Python directly:

```
/mnt/c/Users/<you>/ppk2-venv/Scripts/python.exe measure.py
```

`toggle_DUT_power("ON")` is what powers your board; `get_samples()` returns a pair (samples, digital bits). The README example skips both

Five traps we hit, so you don't have to

1. **Windows programs eat stdin.** Run `python.exe`, `powershell.exe` and flash tools with `</dev/null`, or they swallow the rest of your script
2. **Keep instrument runs in the foreground.** A Windows process started with `&` from a WSL shell dies when that shell exits, and its `finally` (power OFF) never runs
3. **A killed PPK2 run keeps streaming.** The next script then fails with `UnicodeDecodeError`; send `stop_measuring()` and drain the port for half a second before `get_modifiers()`
4. **The PPK2 turns its output off when your script closes its port.** A script that switches power on and exits leaves the board dark; keep a holder process running while you flash and capture
5. **An old** `~/.claude/.credentials.json` can look signed in and still fail; if a prompt errors, run `claude` and sign in again



The bench this guide was tested on. The Windows laptop runs Ubuntu on WSL2, Hapimatic and Claude Code. The black box is the Saleae Logic (ribbon cable), the clear case is the PPK2, and the breadboard holds the test board, with its USB cable back to the laptop. Every step on this page ran here. Lab camera, Oct 26, 2026

Status: beta. Hapimatic's install guide lists Windows as not supported yet; this guide is how we ran it anyway, step by step. Mac: Hapimatic Core is Linux only today. On a Mac, Logic 2's MCP server and ppk2-api still run, so Claude Code or Codex can drive the same loop without Hapimatic (not tested by us). Questions: matt.starfield@solidpd.com

Your boards, measured and flashed

The three boards every team has (Nick's list), how to put the PPK2 on each, and how to flash each from WSL

Board	Put the PPK2 on it	Flash it from WSL (Windows owns USB; call the Windows tool)
Nordic nRF7002 DK nRF5340 + nRF7002	Pull the jumper cap off **P23** (VBAT). PPK2 **VOUT** to P23 pin 1, the upper pin, the side that feeds the chip; **GND** to P21 pin 1 (−); read the pin-1 marker on the silkscreen before clipping, because the lower P23 pin is the DK's own VBAT supply. Source meter at 3.6 V (nRF Connect SDK, *Power profiling of nRF7002 DK*). That measures the nRF7002 Wi-Fi chip; the nRF5340 on P22 needs SB16 cut and SB17 soldered first (*nRF7002 DK Hardware* guide §5.4)	The onboard J-Link: `nrfutil.exe device program --firmware merged.hex`, or `west flash` from the nRF Connect terminal. From WSL, call the same `.exe`, as we did with `arduino-cli.exe` **PATTERN TESTED, BOARD NOT YET**
Seeed XIAO BLE Sense nRF52840	Source mode into the **BAT pads** on the back (3.6–3.8 V), USB unplugged, so you measure the board, not the charger. Seeed forum users measured about 2–5 µA in System OFF with a PPK2 this way. An input pin with a pull-up added about 31 µA in one report: check yours	Double-tap reset: the board appears as a USB drive; copy the `.uf2` from `build/zephyr/zephyr.uf2` to it (`cp` to `/mnt/d/` or the drive letter) **NOT YET TESTED**
ST Nucleo WL55JC STM32WL	USB powers the board. Remove jumper **JP1 (I_SoC)** and put the PPK2 in ampere-meter mode across its two pins (ST UM2592): **VIN** on the 3V3 side, **VOUT** on the STM32 side, GND to any board GND. With JP1 off and nothing across it, the STM32 has no power, so the 3V3 side is the pin that reads 3.3 V to GND. ST confirmed a "JP4" mention in the manual is a typo	The onboard ST-LINK: `STM32_Programmer_CLI.exe -c port=SWD -w zephyr.bin 0x08000000 -rst`, or drag the `.bin` onto its USB drive **NOT YET TESTED**

Nick's two closed-loop requirements, as one loop

1. **Write the pass test first**, as numbers with a tolerance: "frames 500 ± 10 ms apart", "sleep floor below 10 µA"
2. **Prompt from the requirement.** One change per cycle
3. **Build and flash** with the commands above
4. **Measure into files:** Saleae export (logic) and PPK2 CSV (current), every run
5. **Prove or revert,** and commit with the numbers in the message

For the low-power feature, capture the PPK2 **before** the change too: the before/after pair is the metric Nick asked for

Zephyr low-power, the first things to check

- `CONFIG_PM_DEVICE=y` and suspend the peripherals you don't use; UART console alone can hold tens to hundreds of µA on nRF
- Logging and the shell keep the UART awake: `CONFIG_SERIAL=n` and `CONFIG_LOG=n` in a low-power build
- nRF: `sys_poweroff()` for System OFF (lowest), `k_sleep()` for System ON idle
- Measure with the debugger detached and USB unplugged where the board allows: both add current

CHECK Confirm each Kconfig name against your nRF Connect SDK version; it changes between releases

No AI plan from the school? Four routes, today

Codex in Hapimatic	Your ChatGPT sign-in (`codex login`) or an OpenAI key; the $100 Codex credit counts. Penn's ChatGPT Edu includes Codex where a school or instructor pays for seats (Wharton's workspace has it for all members)
Gemini CLI in Hapimatic	Only with a **Google AI Studio API key**, added in Settings › Provider Accounts. Google ended free Gemini CLI *sign-in* for individuals on June 18, 2026; key sign-in still works, and AI Studio's Flash models have a free tier **NOT YET TESTED BY US**
Claude Code	A Claude Pro or Max plan, or an Anthropic API key. The path we tested most, including the full loop on this bench
A local model	We tested a local Qwen on this kind of task: alone it failed 8 of 8 runs; with Claude reviewing each step it passed 3 of 3. Good for cheap steps, not alone

Both of Nick's requirements ran on this setup on Oct 26 with Claude Code in WSL, an original Saleae Logic, a PPK2, and a test board. A feature: it wrote the pass test first, flashed, captured, decoded, and committed (every gap 500.458 to 500.462 ms against 500 ± 5). Low power: it measured first, then cut the blink phase from 43.3 mA to 34.0 mA with the blink still at 2 Hz. Board facts on this page come from the vendors' guides named in each row; anything marked not yet tested is how it should work, not something we have run.